

Feuille d'exercices — Affectations / Entrées-Sorties (BTS SIO)

Organigramme + Algorithme + Code Python

Nom : _____

Date : _____

Consigne générale : Pour chaque exercice (sauf mention contraire), vous devez produire :

- un **organigramme** (flowchart) clair et lisible ;
- un **algorithme** en respectant la convention : *Titre / Variables / Début...Fin* ;
- le **code Python** correspondant (indentation correcte).

Remarque : Sur papier, l'organigramme peut être dessiné à la main dans la zone prévue.

2.1.1 Affectations

Exercice 1 — Jeu d'essai

Après les affectations suivantes :

$$\begin{aligned}A &\leftarrow 1 \\B &\leftarrow A + 1 \\A &\leftarrow B + 2 \\B &\leftarrow A + 2 \\A &\leftarrow B + 3 \\B &\leftarrow A + 3\end{aligned}$$

Quelles sont les valeurs finales de **A** et de **B** ?

Travail demandé

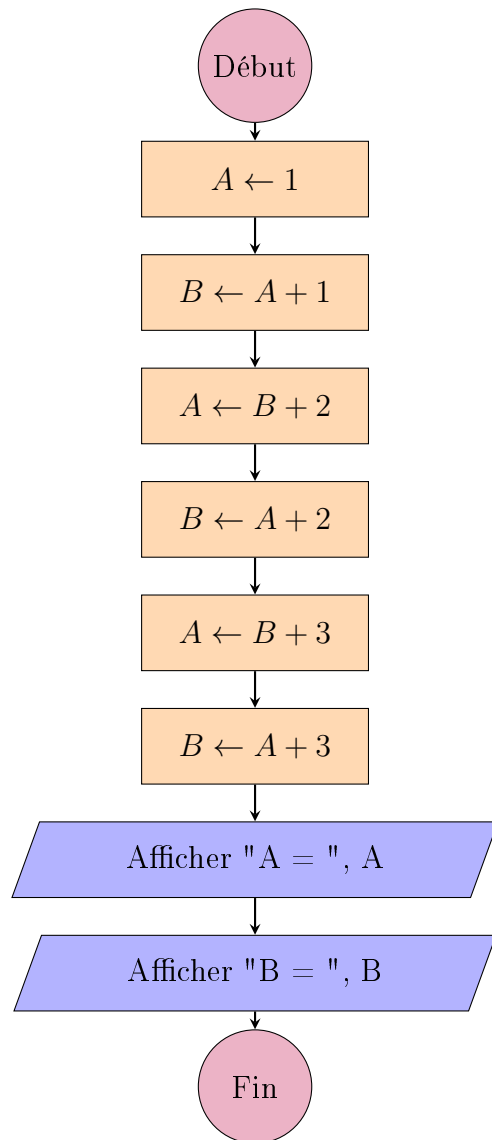
1. Calculez à la main les valeurs finales de A et B .
2. Donnez un organigramme, un algorithme, puis le code Python qui reproduit exactement ces affectations.
3. Affichez à la fin les valeurs de A et B .

Réponse attendue (calcul à la main)

- $A \leftarrow 1$ (A vaut 1)
- $B \leftarrow A + 1$ (B vaut $1+1=2$)
- $A \leftarrow B + 2$ (A vaut $2+2=4$)
- $B \leftarrow A + 2$ (B vaut $4+2=6$)
- $A \leftarrow B + 3$ (A vaut $6+3=9$)
- $B \leftarrow A + 3$ (B vaut $9+3=12$)

Valeurs finales : A = 9, B = 12.

Organigramme



Algorithme

Algorithme JeuEssai

Variables

A, B : Entier

Début

A ← 1

B ← A + 1

A ← B + 2

B ← A + 2

A ← B + 3

B ← A + 3

Afficher "A = ", A

Afficher "B = ", B

Fin

Code Python

```
A = 1
B = A + 1
A = B + 2
B = A + 2
A = B + 3
B = A + 3
print("A =", A)
print("B =", B)
```

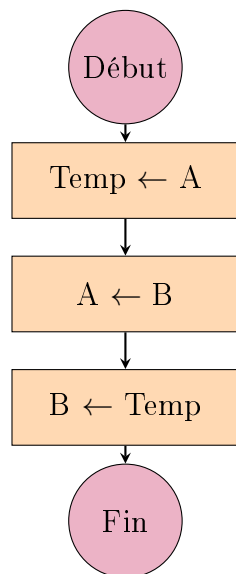
Exercice 2 — Permutation des valeurs de 2 variables

Quelle série d'instructions échange les valeurs des deux variables A et B (déjà initialisées) ?

Travail demandé

1. Proposez un algorithme de permutation en utilisant une **variable temporaire**.
2. Donnez l'organigramme.
3. Donnez le code Python correspondant.

Organigramme



Algorithme

```
Algorithme Permutation
Variables
    A, B, Temp : Entier
Début
    // A et B sont supposées initialisées
    Temp ← A
    A ← B
    B ← Temp
Fin
```

Code Python

```
A = 5
B = 10
print(f"Avant: A={A}, B={B}")
temp = A
A = B
B = temp
print(f"Après: A={A}, B={B}")
```

2.1.2 Saisie, affichage, affectations

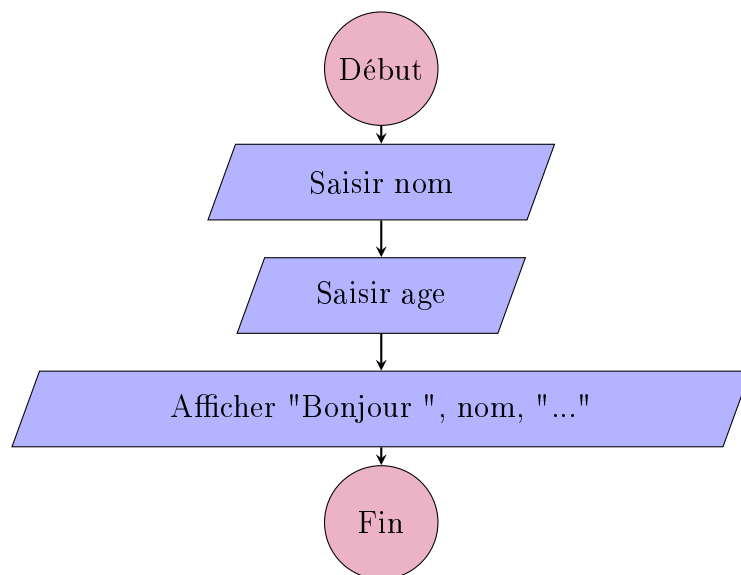
Exercice 3 — Nom et âge

Saisir le nom et l'âge de l'utilisateur et afficher :
"Bonjour ..., tu as ... ans."
en remplaçant les ... par le nom puis l'âge.

Travail demandé

1. Organigramme (entrées : nom, âge ; sortie : phrase).
2. Algorithme (déclarez les types).
3. Python (attention : `input()` renvoie du texte).

Organigramme



Algorithme

Algorithme NomAge

Variables

nom : Chaine

age : Entier

Début

Afficher "Quel est votre nom ?"

Saisir nom

Afficher "Quel est votre âge ?"

Saisir age

Afficher "Bonjour ", nom, ", tu as ", age, " ans."

Fin

Code Python

```
nom = str(input("Quel est votre nom ? "))
age = int(input("Quel est votre âge ? "))
print(f"Bonjour {nom}, tu as {age} ans.")
```

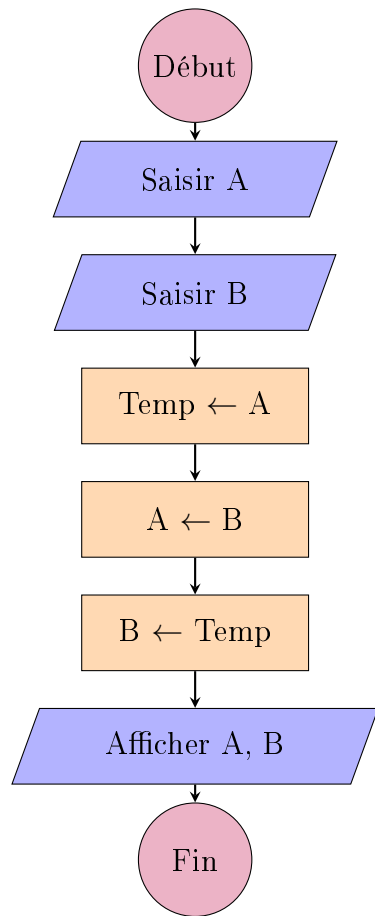
Exercice 4 — Permutation de 2 variables saisies

Saisir deux variables puis **les permuter** avant de les afficher.

Travail demandé

1. Organigramme.
2. Algorithme (avec variable temporaire).
3. Python.

Organigramme



Algorithme

Algorithme PermutationSaisie

Variables

A, B, Temp : Reel

Début

Afficher "Saisir la valeur de A :"

Saisir A

Afficher "Saisir la valeur de B :"

Saisir B

Temp ← A

A ← B

B ← Temp

Afficher "Après permutation: A=", A, ", B=", B

Fin

Code Python

```
A = int(input("Saisir la valeur de A : "))
B = int(input("Saisir la valeur de B : "))
print(f"Avant: A={A}, B={B}")
A, B = B, A
print(f"Après: A={A}, B={B}")
```

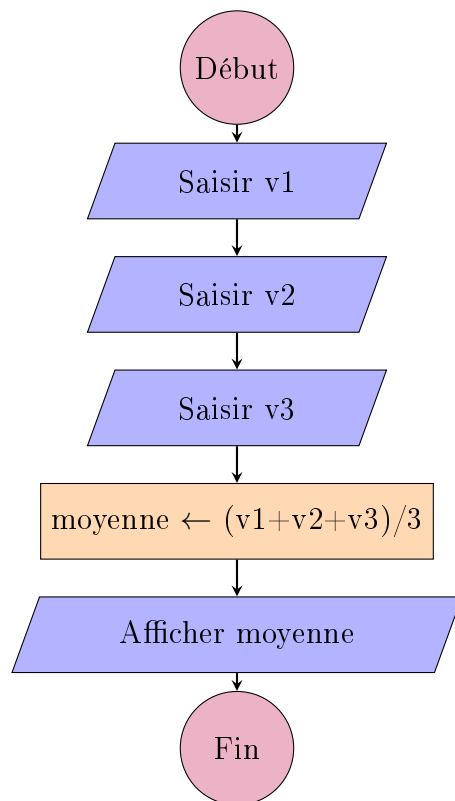
Exercice 5 — Moyenne de 3 valeurs

Saisir 3 valeurs, puis afficher leur moyenne.

Travail demandé

1. Organigramme.
2. Algorithme.
3. Python (choisissez le bon type : entier ou réel).

Organigramme



Algorithme

Algorithme Moyenne

Variables

v1, v2, v3, moyenne : Reel

Début

Afficher "Saisir trois valeurs :"

Saisir v1, v2, v3

moyenne <- (v1 + v2 + v3) / 3

Afficher "La moyenne est : ", moyenne

Fin

Code Python

```
v1 = float(input("Valeur 1: "))
v2 = float(input("Valeur 2: "))
v3 = float(input("Valeur 3: "))
moyenne = (v1 + v2 + v3) / 3
print("La moyenne est :", moyenne)
```

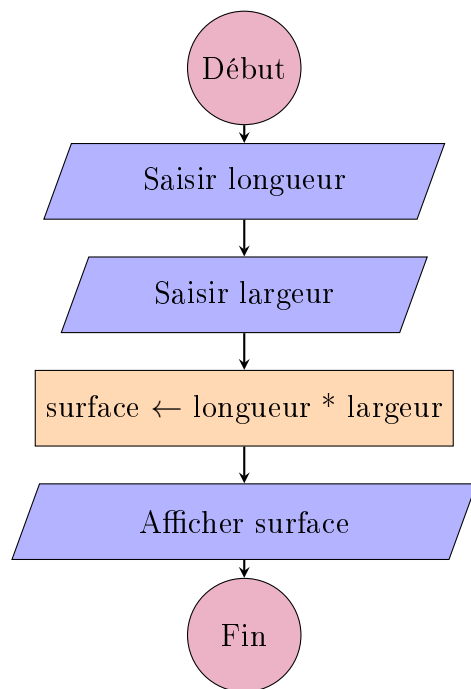
Exercice 6 — Aire du rectangle

Demander à l'utilisateur de saisir la longueur et la largeur d'un rectangle, puis afficher sa surface.

Travail demandé

1. Organigramme.
2. Algorithme.
3. Python.

Organigramme



Algorithme

Algorithme AireRectangle

Variables

longueur, largeur, surface : Reel

Début

Afficher "Saisir la longueur :"

Saisir longueur

Afficher "Saisir la largeur :"

Saisir largeur

surface $\leftarrow$ longueur * largeur

Afficher "La surface est : ", surface

Fin

Code Python

```
longueur = float(input("Saisir la longueur : "))
largeur = float(input("Saisir la largeur : "))
surface = longueur * largeur
print("La surface du rectangle est :", surface)
```

Exercice 7 — Permutation de 4 valeurs

Écrire un algorithme demandant à l'utilisateur de saisir 4 valeurs A, B, C, D et qui permute :

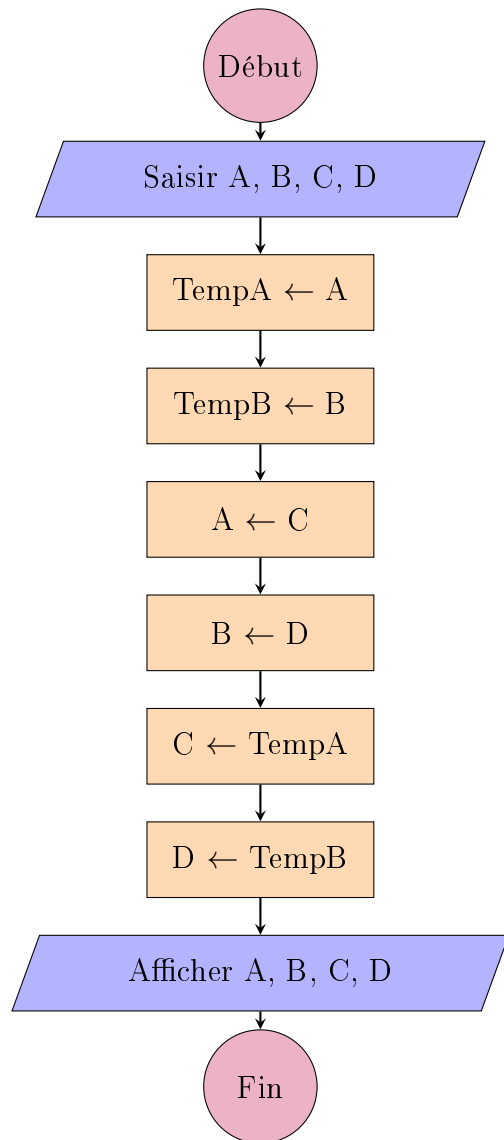
$$(A, B, C, D) \longrightarrow (C, D, A, B)$$

Exemple : si l'utilisateur saisit 1, 2, 3, 4 alors on obtient 3, 4, 1, 2.

Travail demandé

1. Organigramme.
2. Algorithme (utilisez des temporaires si nécessaire).
3. Python.

Organigramme



Algorithme

Algorithme Permutation4_CDBA

Variables

A, B, C, D, TempA, TempB : Reel

Début

Saisir A, B, C, D

TempA ← A

TempB ← B

A ← C

B ← D

C ← TempA

D ← TempB

Afficher A, B, C, D

Fin

Code Python

```
A, B, C, D = 1, 2, 3, 4 # Exemple
print(f"Avant: {(A,B,C,D)}")
```

Exercice 8 — Permutation de 5 valeurs

On considère la permutation :

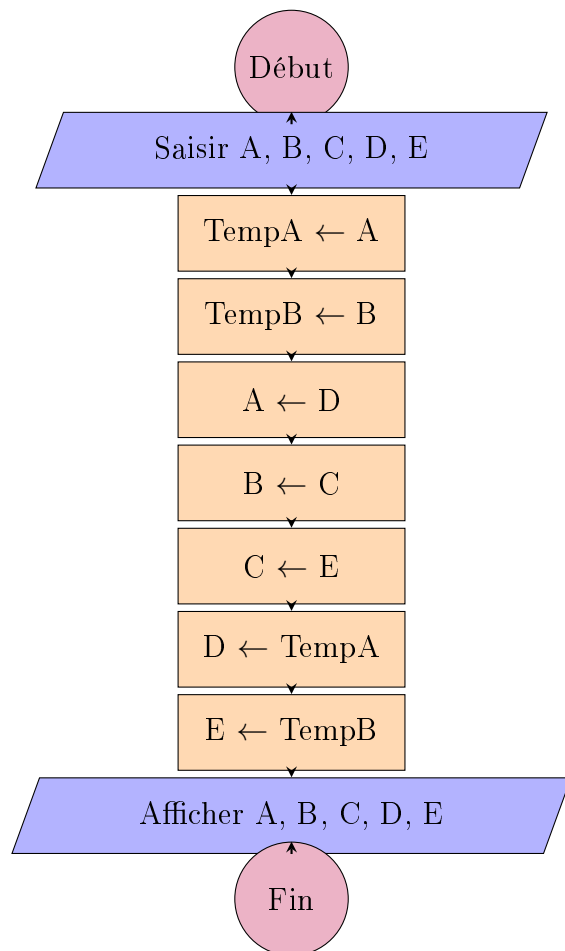
$$(A, B, C, D, E) \longrightarrow (D, C, E, A, B)$$

Exemple : $1, 2, 3, 4, 5 \rightarrow 4, 3, 5, 1, 2$.

Travail demandé

1. Organigramme.
2. Algorithme.
3. Python.

Organigramme



Algorithme

Algorithme Permutation5_DCEAB

Variables

A, B, C, D, E, TempA, TempB : Reel

Début

Saisir A, B, C, D, E

TempA $\leftarrow$ A

TempB $\leftarrow$ B

A $\leftarrow$ D

B $\leftarrow$ C

C $\leftarrow$ E

D $\leftarrow$ TempA

E $\leftarrow$ TempB

Afficher A, B, C, D, E

Fin

Code Python

```
A,B,C,D,E = 1,2,3,4,5 # Exemple
print(f"Avant: {(A,B,C,D,E)}")
A, B, C, D, E = D, C, E, A, B
print(f"Après: {(A,B,C,D,E)}")
```

Exercice 9 — Permutation ultime (6 valeurs)

Même exercice avec :

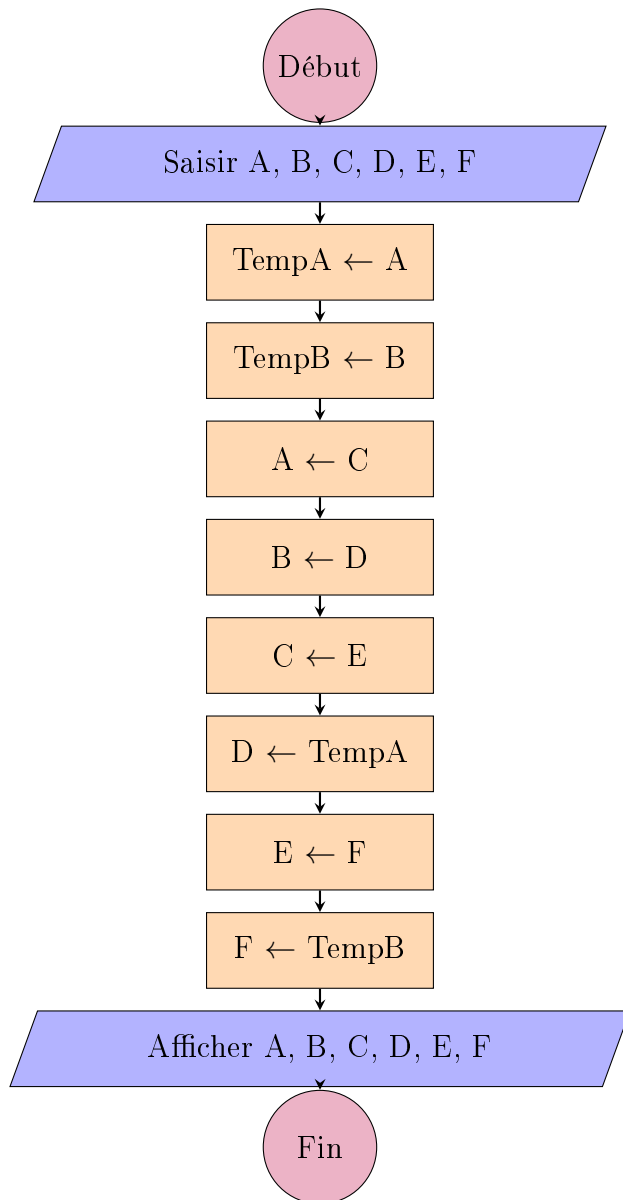
$$(A, B, C, D, E, F) \longrightarrow (C, D, E, A, F, B)$$

Exemple : $1, 2, 3, 4, 5, 6 \rightarrow 3, 4, 5, 1, 6, 2$.

Travail demandé

1. Organigramme.
2. Algorithme.
3. Python.

Organigramme



Algorithme

Algorithme Permutation6_CDEAFB

Variables

A,B,C,D,E,F,TempA,TempB : Reel

Début

Saisir A, B, C, D, E, F

TempA <- A

TempB <- B

A <- C

B <- D

C <- E

D <- TempA

E <- F

F <- TempB

Afficher A, B, C, D, E, F

Fin

Code Python

```
A,B,C,D,E,F = 1,2,3,4,5,6 # Exemple
print(f"Avant: {(A,B,C,D,E,F)}")
A, B, C, D, E, F = C, D, E, A, F, B
print(f"Après: {(A,B,C,D,E,F)}")
```

Exercice 10 — Pièces de monnaie (rendu de monnaie)

On dispose d'un nombre illimité de pièces de **0,50 ; 0,20 ; 0,10 ; 0,05 ; 0,02 ; 0,01** euros. Pour une somme S , on veut payer avec un **nombre minimal de pièces**. Exemple : 0,96 euro $\rightarrow 1 \times 0,50, 2 \times 0,20, 1 \times 0,05, 1 \times 0,01$.

Travail demandé

1. Expliquez (sans formalisme excessif) l'idée intuitive qui justifie qu'on prend toujours la plus grande pièce possible, puis on continue avec le reste.
2. Écrire un algorithme demandant à l'utilisateur de saisir une valeur $S \geq 0$, puis afficher le détail des pièces (quantités) pour payer S avec un minimum de pièces.

Conseil important : Pour éviter les erreurs de virgule, travaillez en **centimes** (entier) :

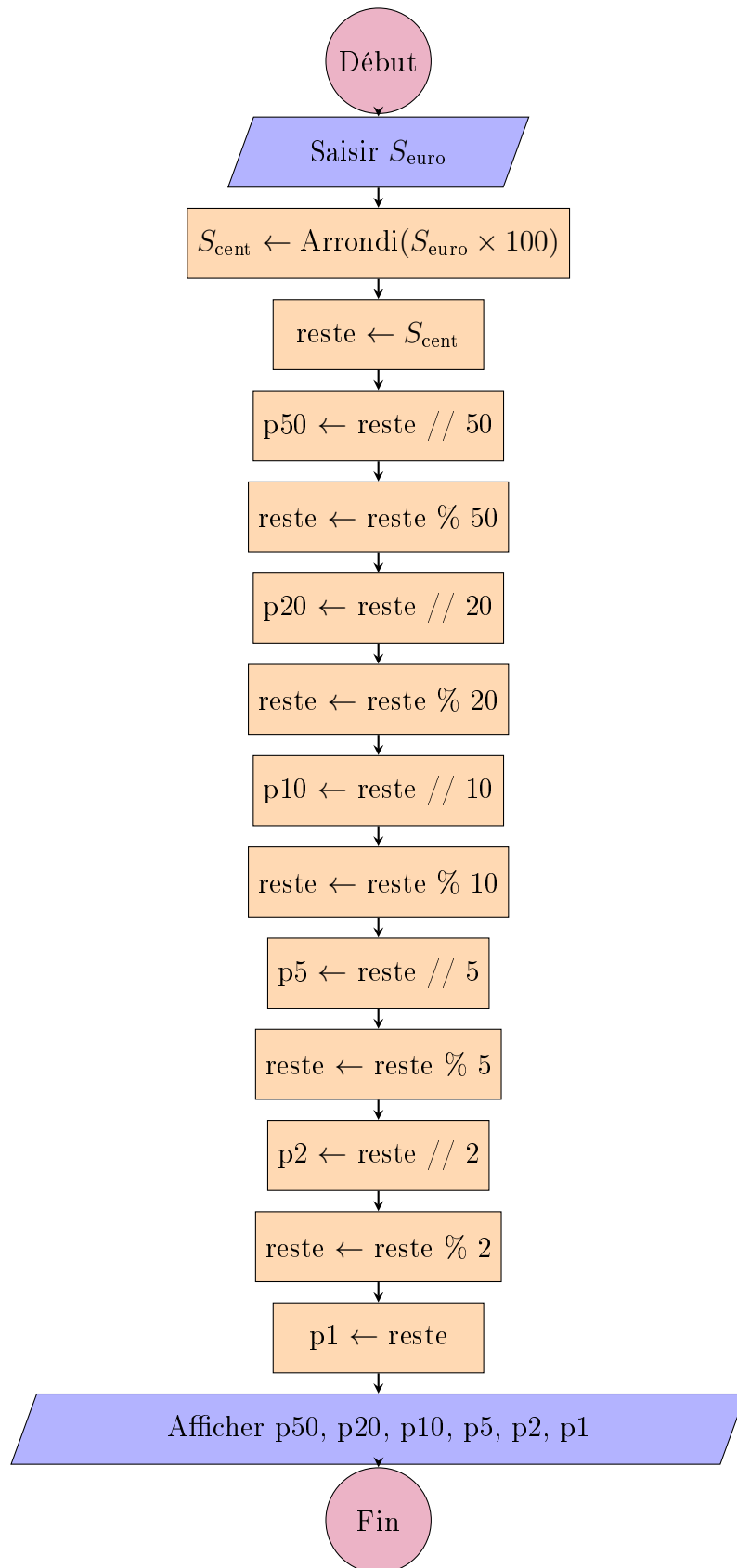
$$S_{\text{cent}} = \text{arrondi}(100 \times S)$$

Puis utilisez des pièces en centimes : 50, 20, 10, 5, 2, 1.

Explication (idée intuitive)

Pour minimiser le nombre total de pièces, il faut maximiser la valeur de chaque pièce choisie. C'est un algorithme "glouton". On prend la plus grande pièce possible (50c) autant de fois que possible. Avec le reste, on recommence avec la pièce la plus grande suivante (20c), et ainsi de suite. Ce système fonctionne car les valeurs des pièces sont choisies pour permettre de former n'importe quelle somme de manière optimale avec cette méthode.

Organigramme



Algorithme

Algorithme RenduMonnaie

Variables

S_euro : Reel
S_cent, reste : Entier
p50, p20, p10, p5, p2, p1 : Entier

Début

Afficher "Saisir la somme S en euros :"
Saisir S_euro

S_cent <- Arrondi(S_euro * 100)
reste <- S_cent

p50 <- reste DIV 50
reste <- reste MOD 50
p20 <- reste DIV 20
reste <- reste MOD 20
p10 <- reste DIV 10
reste <- reste MOD 10
p5 <- reste DIV 5
reste <- reste MOD 5
p2 <- reste DIV 2
p1 <- reste MOD 2

Afficher S_cent, " centimes ="
Afficher p50,"x50c, ",p20,"x20c, ",p10,"x10c, "
Afficher p5,"x5c, ",p2,"x2c, ",p1,"x1c"

Fin

Code Python

```
import math

s_euro = float(input("Entrez la somme en euros: "))
s_cent = int(round(s_euro * 100))
reste = s_cent

pieces = [50, 20, 10, 5, 2, 1]
rendu = {}

for p in pieces:
    nb_pieces = reste // p
    if nb_pieces > 0:
        rendu[p] = nb_pieces
    reste %= p

print(f"{s_cent} centimes se décompose en :")
for p, nb in rendu.items():
    print(f"- {nb} pièce(s) de {p} centimes")
```